action. It will transform these action values into probabilities. Then we use the `np.random.choice` function, which accepts an arbitrary input array, *x*, and a parameter, *p*, that is an array of probabilities that correspond to each element in *x*. Since our record is initialized to all zeros, `softmax` at first will return a uniform distribution over all the arms, but this distribution will quickly skew toward whatever action is associated with the highest reward. Here's an example of using `softmax` and the random choice function:

```
>>> x = np.arange(10)
>>> x
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> av = np.zeros(10)
>>> p = softmax(av)
>>> p
array([0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1])
>>> np.random.choice(x,p=p)
3
```

We used the numpy `arange` function to create an array from 0 to 9, corresponding to the indices of each arm, so the random choice function will return an arm index according to the supplied probability vector. We can use the same training loop as we did previously; we just need to change the arm selection part so it uses `softmax` instead of `get_best_arm`, and we need to get rid of the random action selection that's part of the epsilon-greedy strategy.

Listing 2.8. Softmax action-selection for the *n*-armed bandit

```
n = 10
probs = np.random.rand(n)
record = np.zeros((n,2))

fig,ax = plt.subplots(1,1)
ax.set_xlabel("Plays")
ax.set_ylabel("Avg Reward")
fig.set_size_inches(9,5)
rewards = [0]
for i in range(500):
    p = softmax(record[:,1])                          1
    choice = np.random.choice(np.arange(n),p=p)       2
    r = get_reward(probs[choice])
    record = update_record(record,choice,r)
    mean_reward = ((i+1) * rewards[-1] + r)/(i+2)
```